Royaume du Maroc
Université Sidi Mohamed Ben Abdellah
**École Nationale des Sciences Appliquées de Fès**

Filière : Génie des Systèmes Communicants et Sécurité Informatique (GSCSI)

## Rapport de stage de Projet de Fin d'Année

# `demo_cli`

Un garde-fou pour agents de code IA :
prédire l'effet d'une commande, récupérer, tracer

Réalisé par

**Nizar Yartaoui**

Encadrant académique
**Mme K. Chetioui**
ENSA Fès

Encadrant professionnel
**M. Younes Lakhouane**
WePwn

Organisme d'accueil : **WePwn**

Période du stage : du 15 juin 2026 au **[À COMPLÉTER : date de fin]**

Année universitaire 2025–2026

*[À COMPLÉTER : dédicace : texte personnel de l'auteur, ou supprimer ce fichier de main.tex]*

# Remerciements

Je tiens à remercier **M. Younes Lakhouane**, fondateur de WePwn et mon encadrant professionnel, pour la confiance qu'il m'a accordée en me remettant `demo_cli` en version bêta, et pour la liberté qu'il m'a laissée d'en reprendre l'architecture, de l'attaquer et de la consolider.

J'adresse mes remerciements à **Mme K. Chetioui**, mon encadrante académique à l'ENSA de Fès, pour son suivi et ses conseils tout au long de ce projet.

Je remercie également l'ensemble du corps professoral de la filière Génie des Systèmes Communicants et Sécurité Informatique, ainsi que les membres du jury qui ont accepté d'évaluer ce travail.

**[À COMPLÉTER : relire, compléter ou personnaliser ces remerciements]**

# Résumé

[À COMPLÉTER : résumé rédigé après validation des chapitres]

**Mots-clés :** agents de code IA, sécurité, analyse de commandes shell, simulation, récupération, ptrace, WinFsp, PowerShell, journal chaîné.

# Abstract

[À COMPLÉTER : abstract written after the chapters are approved]

**Keywords:** AI coding agents, security, shell command analysis, simulation, recovery, ptrace, WinFsp, PowerShell, hash-chained ledger.

# Table des matières

**Remerciements** ii

**Résumé** iii

**Abstract** iv

**Liste des abréviations et glossaire** ix

**Introduction générale** 1

**1 Contexte général du projet** 2

1.1 L'organisme d'accueil : WePwn . . . 2

1.1.1 Présentation . . . 2

1.1.2 Fiche signalétique . . . 3

1.1.3 Cadre du stage . . . 3

1.2 Point de départ et contribution . . . 3

1.2.1 La version bêta remise au début du stage . . . 3

1.2.2 Ce que le stage a apporté . . . 5

1.3 Problématique : des agents qui exécutent des commandes . . . 5

1.3.1 Un agent n'écrit plus seulement du code, il l'exécute . . . 5

1.3.2 Trois incidents publics . . . 6

1.3.3 Pourquoi le problème est difficile . . . 6

1.3.4 Énoncé du problème . . . 7

1.4 Objectifs du projet . . . 7

1.5 Méthodologie de travail . . . 7

1.5.1 Le risque propre à un outil de sécurité . . . 7

1.5.2 Une partie à la fois . . . 8

1.5.3 Les portes d'un atterrissage . . . 9

1.5.4 Le message de commit comme compte rendu . . . 10

1.6 Planning . . . . . 10
1.6.1 Déroulement du stage . . . . . 10
1.6.2 Chronologie des parties documentées . . . . . 10

**2 État de l'art et étude de l'existant 13**

**3 Analyse et conception 14**
3.1 Modèle de menace et invariant . . . . . 14
3.2 Architecture d'ensemble . . . . . 14
3.3 Conception du moteur . . . . . 15
3.4 La politique et le décideur unique . . . . . 15
3.5 Deux systèmes, un décideur . . . . . 15

**4 Réalisation 19**
4.1 Le côté Windows et C3, le second frontal du moteur . . . . . 19

**5 Sécurité, validation et limites 20**

**Conclusion générale et perspectives 21**

**Bibliographie et webographie 22**

**A Exemples de reçus 23**

**B Extrait de la table de règles 24**

**C Extraits de tests 25**

**D Captures d'écran 26**

# Table des figures

1.1 La méthode : le cycle d'une partie et les portes à franchir avant l'atterrissage . 9

1.2 Chronologie des parties livrées, d'après l'historique git (21 septembre – 9 octobre 2026) . . . . . 11

3.1 Le flux d'ensemble : de la commande de l'agent au reçu . . . . . 14

3.2 Ce que l'outil voit : chaque couche et ce qu'elle couvre, selon l'endroit où la commande agit . . . . . 15

3.3 L'intérieur du moteur, sur un exemple réel (sortie du moteur au commit `3c94414`) 16

3.4 La table de décision : où l'effet tombe × quelle sorte d'effet . . . . . 17

3.5 Linux et Windows : un seul décideur, deux filets d'exécution . . . . . 18

4.1 C3 : deux frontaux, un seul moteur (d'après le brief C3 ; statut : en cours de finalisation) . . . . . 19

# Liste des tableaux

1.1 Fiche signalétique de WePwn . . . . . 3
1.2 De la version bêta à l'état actuel (commit `3c94414`, 9 octobre 2026) . . . . . 4
1.3 Lignes où une lecture naïve se trompe (extraits de `tests/test_command_model.py` et des commits I1a, W0) . . . . . 7
1.4 Objectifs du stage et critère de vérification de chacun . . . . . 8

# Liste des abréviations et glossaire

## Abréviations

| Sigle | Signification |
|---|---|
| ACL | *Access Control List* — liste de contrôle d'accès (Windows/NTFS) |
| API | *Application Programming Interface* |
| AST | *Abstract Syntax Tree* — arbre syntaxique abstrait |
| CLI | *Command-Line Interface* — interface en ligne de commande |
| CPU | *Central Processing Unit* |
| ENSAF | École Nationale des Sciences Appliquées de Fès |
| FS | *File System* — système de fichiers |
| FUSE | *Filesystem in Userspace* (Linux) |
| GSCSI | Génie des Systèmes Communicants et Sécurité Informatique |
| HTTP | *HyperText Transfer Protocol* |
| IA | Intelligence artificielle |
| IaC | *Infrastructure as Code* (Terraform, Pulumi…) |
| LLM | *Large Language Model* — grand modèle de langage |
| PaaS | *Platform as a Service* |
| PFA | Projet de Fin d'Année |
| PITR | *Point-In-Time Recovery* — restauration à un instant donné |
| POSIX | *Portable Operating System Interface* — ici, le langage des shells `sh`/`bash` |
| SaaS | *Software as a Service* |
| SDDL | *Security Descriptor Definition Language* (Windows) |
| SQL | *Structured Query Language* |
| UAC | *User Account Control* — élévation de privilèges sous Windows |
| VCS | *Version Control System* — gestion de versions (git) |

| Sigle | Signification |
|---|---|
| WSL | *Windows Subsystem for Linux* |

# Glossaire

**Agent de code IA**
: Programme fondé sur un LLM qui lit et modifie un projet et exécute des commandes shell pour le compte d'un développeur (Claude Code, Codex…).

**Porte**
: Point par lequel une action de l'agent atteint `demo_cli` avant de s'exécuter : *hook* de l'agent, porte d'édition de fichiers, garde du shell, commande `demo_cli check`.

**Effet**
: Ce qu'une commande fait au monde, dans un vocabulaire commun : DELETE, OVERWRITE, TRUNCATE, MOVE, CREATE, APPEND, RUN, SEND, LATER…

**Surface distante**
: Ce qu'aucune copie locale ne peut restaurer : base de données distante, historique git d'un serveur, ressource cloud, conteneur, message envoyé…

**Non couvert**
: (*uncovered*) Un outil que la table de règles ne décrit pas encore.

**Non compris**
: (*not understood*) Une partie de ligne que le moteur ne peut pas lire avant l'exécution (valeur inconnue, construction non modélisée).

**Point de récupération**
: Copie, prise *avant* l'exécution, de ce que la commande va détruire ; `demo_cli undo <id>` la restaure.

**Filet d'exécution**
: Mécanisme qui observe la commande *pendant* qu'elle tourne et sauvegarde chaque fichier au moment où il est détruit : garde d'appels système (Linux), montage WinFsp (Windows).

**Plan de contrôle**
: Les fichiers de `demo_cli` lui-même (configuration, registre, réserve de récupération, configuration des *hooks*) : un agent ne peut pas les modifier.

**Reçu**
: Entrée du registre décrivant une décision ; chaque reçu contient l'empreinte du précédent (chaîne SHA-256).

**Portail de décision**

Comparaison, sur un corpus de 922 commandes, de chaque décision avant et après une modification.

**Arbitrage**

(*ruling*) Décision écrite et datée qui tranche un cas de politique avant son implémentation.

# Introduction générale

[À COMPLÉTER : introduction générale, rédigée une fois les chapitres validés]

CHAPITRE 1

# Contexte général du projet

Ce chapitre situe le travail. Il présente l'organisme d'accueil, WePwn, puis le point de départ concret du stage : un outil, `demo_cli`, remis en version bêta. Il pose ensuite le problème que cet outil traite — des agents d'intelligence artificielle qui exécutent eux-mêmes des commandes sur la machine d'un développeur —, fixe les objectifs, et décrit la méthode de travail qui a permis de faire évoluer un logiciel de sécurité sans jamais le rendre, même un instant, moins sûr.

## 1.1 L'organisme d'accueil : WePwn



**WePwn**

Sécurité offensive · programmes de *bug bounty* · divulgation coordonnée de vulnérabilités · outils de sécurité

### 1.1.1 Présentation

WePwn est une entreprise marocaine de sécurité offensive. Son activité d'origine est le *bug bounty* : une entreprise ouvre un programme, des chercheurs en sécurité indépendants testent ses services, et chaque vulnérabilité validée est récompensée. WePwn se présente comme la mise en relation entre des entreprises et des chercheurs, afin que les premières soient averties des vulnérabilités identifiées chez elles [1]. Selon l'organisme, elle a été la première plateforme de *bug bounty* au Maroc [1] ; les programmes qu'elle héberge rémunèrent les chercheurs en dirhams. Un portail dédié permet aux chercheurs de rejoindre ces programmes [2].

Autour de cette activité, WePwn propose un service de divulgation de vulnérabilités géré de bout en bout [3] : tri des signalements, évaluation de la sévérité, validation, rapport détaillé, conseils de correction et pilotage du programme dans la durée. Ce métier — recevoir des attaques réelles, les reproduire, en mesurer l'impact et accompagner la correction — a naturellement conduit l'entreprise vers un second axe : les *pratiques* et les *outils* de sécurité. `demo_cli` appartient à cet axe.

1. Affirmation de l'organisme d'accueil, reprise telle quelle.

### 1.1.2 Fiche signalétique

**Table 1.1.** Fiche signalétique de WePwn

| | |
|---|---|
| Raison sociale | WePwn |
| Domaine | Cybersécurité — sécurité offensive |
| Activités | Programmes de *bug bounty* ; divulgation de vulnérabilités gérée ; pratiques et outils de sécurité |
| Fondateur | M. Younes Lakhouane |
| Site web | `https://wepwn.ma` |
| Siège | **[À COMPLÉTER : ville]** |
| Année de création | **[À COMPLÉTER : année]** |
| Effectif | **[À COMPLÉTER]** |

### 1.1.3 Cadre du stage

Le stage s'est déroulé au sein de WePwn, sous l'encadrement professionnel de M. Younes Lakhouane, fondateur de l'entreprise, et sous l'encadrement académique de Mme K. Chetioui (ENSA de Fès). Il a commencé le 15 juin 2026. Le sujet porte sur `demo_cli`, un outil de WePwn publié en source ouverte sous licence MIT [4], que l'entreprise destine aux équipes qui confient du travail à des agents de code IA.

La double culture de l'entreprise explique la forme du travail demandé : construire l'outil *et* l'attaquer. Une partie du stage a consisté à développer et à architecturer ; une autre, à chercher comment contourner ce qui venait d'être construit, avec les méthodes du *bug bounty* — lire le code, fabriquer des commandes piégées, fouiller les traces. Le chapitre 5 rend compte de cette seconde partie.

## 1.2 Point de départ et contribution

### 1.2.1 La version bêta remise au début du stage

Le stage ne part pas d'une page blanche. WePwn avait publié `demo_cli` en version 0.4.0-beta.8 (19 juillet 2026) [4]. Cette bêta posait déjà l'idée qui reste au cœur du projet, et qu'il faut attribuer à son point d'origine :

**L'invariant hérité de la bêta**

Une action qui modifie quelque chose doit être récupérable *et* conforme au contexte déclaré ; sinon elle est escaladée — jamais autorisée en silence, jamais présentée à tort comme « récupérée ».

De cet invariant découlait une posture inhabituelle pour un outil de sécurité : *récupérer plutôt que bloquer*. Quand `demo_cli` peut prouver qu'il a copié la cible réelle d'une commande destructrice, il la copie et laisse l'agent continuer ; une commande `demo_cli undo <id>` remet les fichiers en place. Il ne bloque que ce qui ne peut pas être récupéré (un `terraform destroy`, un `git push -force`). La bêta s'intégrait à Claude Code et à Cursor, savait prévisualiser une requête SQL destructrice et laissait de chaque décision un reçu chaîné.

Elle annonçait aussi, honnêtement, ses limites. Son journal des modifications le dit en une phrase : l'analyse des commandes shell repose sur des motifs, « un arbre syntaxique complet est sur la feuille de route ». Concrètement, le classifieur tenait en 340 lignes et 9 expressions régulières. Le tableau 1.2 compare cette bêta à l'état du projet au commit de référence `3c94414`.

**Table 1.2.** De la version bêta à l'état actuel (commit `3c94414`, 9 octobre 2026)

| | **Bêta 0.4.0-beta.8** | **Aujourd'hui** |
|---|---|---|
| Code source (Python) | ≈ 3 700 lignes | ≈ 33 700 lignes |
| Tests | ≈ 1 600 lignes, 11 fichiers | ≈ 28 700 lignes, 133 fichiers |
| Lecture d'une commande | 9 expressions régulières | analyseur POSIX, simulation sur un système de fichiers virtuel, table de 71 règles (212 outils) |
| Décisions | 5 | 6 (+ SUPERVISED), canal « demander / refuser » |
| Commandes illisibles | motifs, au mieux | « non compris » : filet, point de contrôle ou escalade — jamais un ALLOW |
| Pendant l'exécution | rien | garde d'appels système (Linux), montage WinFsp (Windows) |
| Surfaces distantes | quelques motifs | ≈ 40 surfaces, chacune avec sa règle « ask » ou « deny » |
| Agents | Claude Code, Cursor | Claude Code (Bash, PowerShell, fichiers), Codex, Cursor, garde du shell |
| Fichiers de `demo_cli` | non protégés | plan de contrôle refusé à l'agent |
| Validation | tests unitaires | tests + exemples de règles + portail de 922 commandes, Linux et Windows |

### 1.2.2 Ce que le stage a apporté

La mission confiée était de reprendre cette bêta, de la poursuivre et d'en reprendre l'architecture, en menant de front le développement et le travail offensif. Le dépôt de travail conserve l'historique détaillé des 50 derniers commits, du 21 septembre au 9 octobre 2026 ; il commence par l'import d'une version 1.7.0 déjà largement étendue par rapport à la bêta. **[À COMPLÉTER : travaux réalisés entre la bêta de juillet et la version 1.7.0 : nature, dates, où en trouver la trace]**

Sur la période documentée, la contribution se résume en quatre lignes :

1. **Un moteur au lieu de motifs** : une analyse syntaxique unique des lignes POSIX, une simulation de leurs effets et une table de règles écrite comme des données (chapitres 3 et 4).
2. **Une seule politique, un seul décideur** : ce qui était dispersé entre plusieurs modules est réuni dans `policy.py` et `decide.py`.
3. **Les deux systèmes** : chaque partie est conçue et vérifiée sous Linux et sous Windows ; le filet d'exécution existe sur les deux.
4. **Le travail offensif** : un audit pinçant 18 défauts, puis une suite de vulnérabilités (V1–V20) trouvées et corrigées, chacune avec un test qui échoue sur le code d'avant (chapitre 5).

Une part essentielle du rôle a été celle d'*arbitre*. Chaque cas de politique délicat — faut-il autoriser un `git reset` que le *reflog* sait annuler ? demander ou refuser la suppression d'un volume Docker ? — a fait l'objet d'une décision écrite, numérotée et datée avant d'être codée. Les messages de commit et les tests en gardent la trace : « ruling (c) », « rulings M1–M5 », « Nizar's table, 2026-10-07 ».

## 1.3 Problématique : des agents qui exécutent des commandes

### 1.3.1 Un agent n'écrit plus seulement du code, il l'exécute

Un agent de code IA comme Claude Code ou Codex ne se contente pas de proposer du texte. Il lit le projet, modifie des fichiers, et surtout lance des commandes dans un shell — `bash` sous Linux, PowerShell sous Windows — pour compiler, tester, nettoyer, migrer une base, déployer. Chaque commande s'exécute avec les droits du développeur, sur sa machine, souvent à côté de données qui n'existent nulle part ailleurs. L'agent se trompe parfois ; une erreur dans une commande destructrice n'est pas une suggestion maladroite, c'est une perte de données.

### 1.3.2 Trois incidents publics

Le risque n'est pas théorique. Trois cas documentés illustrent trois formes du problème.

**Une base de production supprimée (Replit, juillet 2025)**

Au cours d'une expérience de plusieurs jours, l'agent de Replit a exécuté des commandes qui ont effacé la base de données de production d'un utilisateur, alors qu'il avait reçu la consigne de ne rien modifier sans accord explicite ; il a ensuite affirmé à tort que la restauration était impossible [5, 6]. *Leçon : une consigne n'est pas une protection, et l'agent n'est pas une source fiable sur ce qui peut être récupéré.*

**Des fichiers écrasés par des déplacements (Gemini CLI, juillet 2025)**

L'agent a cru avoir créé un dossier dont la création avait échoué, puis y a « déplacé » les fichiers de l'utilisateur ; chaque déplacement vers un nom inexistant a écrasé le précédent, et tous les fichiers sauf un ont été perdus [7, 8]. *Leçon : aucune commande de la séquence ne « ressemble » à une suppression ; seul un raisonnement sur l'état du système de fichiers révèle la perte.*

**Des fichiers supprimés en voulant les compter (Claude Code, juillet 2026)**

Chargé de compter des fichiers de rapport, un agent a lancé une commande qui les a supprimés ; l'utilisateur en a récupéré une partie avec un outil de récupération de données [9]. La bêta de `demo_cli` cite ce cas comme motivation. *Leçon : demander une confirmation ne suffit pas quand la personne fait confiance à l'intention annoncée ; ce qui manquait, c'était une copie prise avant.*

### 1.3.3 Pourquoi le problème est difficile

Juger une commande *avant* qu'elle s'exécute paraît simple : repérer `rm`, et se méfier. En réalité le langage du shell rend cette lecture trompeuse. Le tableau 1.3 montre quatre lignes, tirées des tests du projet, où une lecture naïve — par motifs, ou par un analyseur trop confiant — se trompe dans un sens ou dans l'autre.

À cela s'ajoutent les enveloppes (`sudo`, `xargs`, `timeout`), les commandes lancées par du code que l'outil ne lit pas (`python script.py`), les effets à distance (une base, un dépôt git distant, un bucket) qu'aucune copie locale ne restaure, et un second langage, PowerShell, où les arguments se lient aux paramètres par position.

**Table 1.3.** Lignes où une lecture naïve se trompe (extraits de `tests/test_command_model.py` et des commits I1a, W0)

| Ligne | Lecture naïve | Ce que fait le shell |
|---|---|---|
| `ls # ; rm -rf x` | un `rm -rf` | rien : tout ce qui suit # est un commentaire |
| `echo $(date; rm -rf x) >/dev/null` | un `echo` | exécute `rm -rf x` dans la substitution |
| `bash -o pipefail -c "rm app.db"` | une option inconnue, rien à juger | supprime `app.db` |
| `X=-rf; rm $X src` | un `rm` sans option | un `rm -rf src` |

### 1.3.4 Énoncé du problème

**Problématique**

Comment, avant qu'une commande écrite par un agent ne s'exécute, en prédire l'effet assez exactement pour laisser passer ce qui est sans danger, rendre récupérable ce qui peut l'être, et arrêter ou soumettre à une personne le reste — sans jamais revendiquer une protection qu'on n'a pas prise, et de la même manière sous Linux et sous Windows ?

## 1.4 Objectifs du projet

Les objectifs découlent de la problématique ; chacun est formulé de façon à pouvoir être vérifié dans le code et les tests.

Un objectif est volontairement *absent* : faire de `demo_cli` une barrière contre un adversaire déterminé. L'outil protège contre les *erreurs* d'un agent coopératif ; ce modèle de menace, hérité de la bêta et affiché par `demo_cli doctor`, est discuté à la section 3.1.

## 1.5 Méthodologie de travail

### 1.5.1 Le risque propre à un outil de sécurité

Faire évoluer un outil de sécurité comporte un risque particulier : une modification qui corrige un cas peut, sans bruit, en autoriser un autre. Un test « vert » ne le révèle pas si aucun test ne décrivait le cas devenu permissif. L'audit du 26 septembre l'a montré : la suite passait avec 1856 tests réussis alors que 18 défauts confirmés restaient atteignables, parce qu'aucun test ne décrivait ces comportements. La méthode adoptée répond directement à ce constat.

**Table 1.4.** Objectifs du stage et critère de vérification de chacun

| | Objectif | Comment on le vérifie |
|---|---|---|
| O1 | **Prédire** l'effet d'une ligne à partir de sa structure, pas de son texte. | Chaque exemple de la table de règles (763) est un test qui compare l'impact prédit à l'impact attendu. |
| O2 | **Ne jamais autoriser ce qui n'est pas lu.** Une ligne illisible n'obtient pas ALLOW. | Tests M2.3a : une ligne « non comprise » va au filet, à un point de contrôle, sinon elle est escaladée. |
| O3 | **Ne jamais revendiquer une récupération non prise.** | Tests des captures partielles (tout ou rien), de V17 (point de contrôle hors projet). |
| O4 | **Voir pendant l'exécution** ce que la lecture ne peut pas prédire. | Garde d'appels système (Linux) et montage WinFsp (Windows) testés sur les deux systèmes. |
| O5 | **Traiter le distant honnêtement** : distinguer « demander à la personne » et « refuser ». | Table `policy.ESCALATION_CHANNEL` ; une surface sans ligne est refusée. |
| O6 | **Les deux systèmes** : Linux et Windows, avec les mêmes garanties. | Suite complète passée sur les deux systèmes avant chaque atterrissage. |
| O7 | **Ne rien rendre plus permissif sans le vouloir.** | Portail de décision : 922 commandes comparées avant et après chaque partie. |
| O8 | **Résister à l'agent** : il ne doit pas pouvoir désarmer l'outil. | Plan de contrôle refusé à toutes les portes ; tâche Windows durcie (W-SEC). |

### 1.5.2 Une partie à la fois

Le travail est découpé en *parties* courtes et nommées (A, B, C1, SYS1, M2.2b3, W-SEC…). Chaque partie suit le même cycle (figure 1.1) :

1. **Plan** : périmètre, cas à couvrir, risques, ce qui ne doit pas bouger.
2. **Arbitrages** : chaque question de politique est tranchée *par écrit* avant d'être codée, avec un identifiant et une date (« ruling R2 : un envoi qui peut remplacer un objet distant est escaladé »). Le code et ses tests citent l'arbitrage.
3. **Prototype** : le code, et des tests qui *échouent sur le code d'avant*. Un test qui passerait aussi sur l'ancien code ne prouve rien sur la correction.
4. **Revue** : relecture adverse et sondes — des commandes fabriquées pour piéger la nouvelle logique, lancées dans un projet jetable.
5. **Atterrissage** : un commit dont le message est un bilan : ce qui change, les arbitrages appliqués, les tests ajoutés, et le résultat du portail de décision.

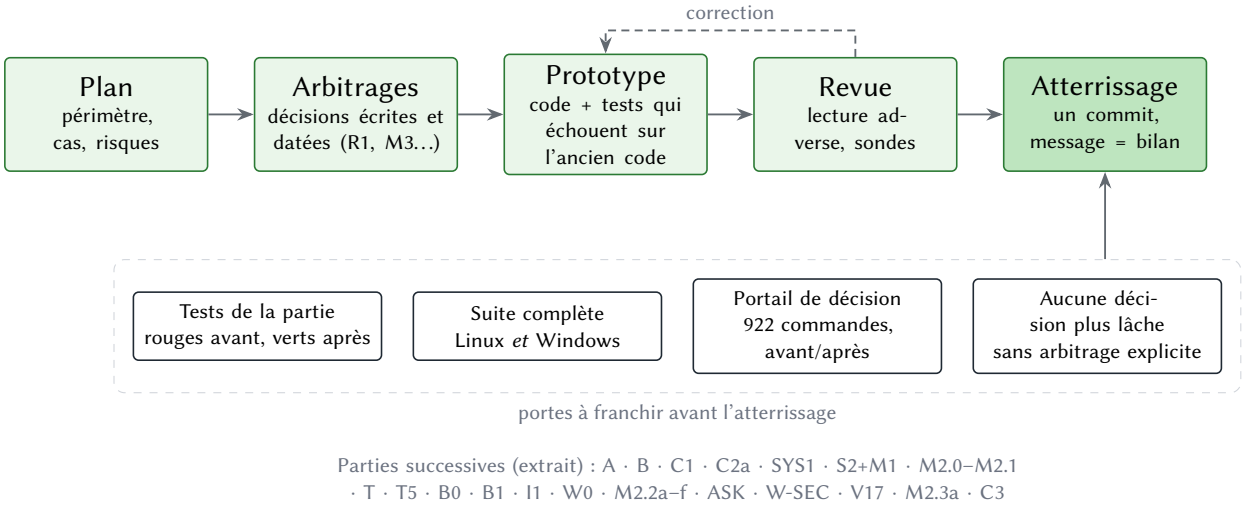



**Figure 1.1.** La méthode : le cycle d'une partie et les portes à franchir avant l'atterrissage

### 1.5.3 Les portes d'un atterrissage

**Les tests de la partie**

Rouges sur le code d'avant, verts après.

**La suite complète, sous Linux et sous Windows**

Les deux systèmes n'ont pas les mêmes chemins (`C:\`, `/c/` sous Git Bash), ni les mêmes appels système, ni le même shell : un test vert sous Linux ne dit rien de Windows.

**Le portail de décision**

Un corpus de 922 commandes réelles et fabriquées est évalué avant et après la modification ; toute ligne dont la décision, la règle, la cible ou la capture change est listée et doit être justifiée. Les messages de commit en rendent compte (« the 922 gate changes 124 reviewed decisions : 8 fixed misses, 18 newly captured, 86 newly allowed »).

**Aucune décision plus permissive sans arbitrage**

Une ligne qui passe d'ESCALATE à ALLOW sans arbitrage explicite bloque l'atterrissage, même si tous les tests passent.

> **Pourquoi un portail de décision en plus des tests ?**
>
> Les tests vérifient ce qu'on a pensé à écrire. Le portail vérifie *tout le reste* : il ne sait pas quelle décision est juste, mais il voit chaque décision qui change. C'est lui qui transforme « la suite passe » en « rien n'a bougé sans qu'on le sache ».

### 1.5.4 Le message de commit comme compte rendu

Chaque atterrissage laisse un compte rendu lisible directement dans l'historique. L'extrait suivant est réel (les lignes « [...] » sont coupées) :

```
• • •  git show -s 3c94414

$ git show -s --format=%B 3c94414
M2.3a: lines the engine does not fully read leave the old text rules

- runners (python, node, npm, make, ./x, source, awk programs that run or
  write ...) go to the runtime net, else a checkpoint, else ALLOW with a
  note that nothing was captured (ruling M1)
- a line the engine cannot read goes to the net, else a checkpoint, else
  ESCALATE (M3) - unread SQL such as psql -c "$(cat q.sql)" was ALLOWED
  (V19); where the old rules flag the line they keep their capture or
  escalation, never an ALLOW
[...]
- engine rules for gzip / bzip2 / xz, sort -o, iconv -o, patch,
  awk -i inplace, 7z: they deleted or overwrote files unseen (M4, V20)
[...]
- tests/test_m23a.py (17); row fixtures of earlier parts: every moved row
  stricter; gate: 6 rows move, reference re-recorded
```

## 1.6 Planning

### 1.6.1 Déroulement du stage

Le stage a débuté le 15 juin 2026 et s'est terminé le **[À COMPLÉTER : date de fin]**. **[À COMPLÉTER : phases du stage, de la prise en main de la bêta à la fin : à confirmer pour le diagramme de Gantt d'ensemble]**

### 1.6.2 Chronologie des parties documentées

L'historique du dépôt de travail permet de dater précisément les parties livrées entre le 21 septembre et le 9 octobre 2026. La figure 1.2 les regroupe en cinq phases : la reprise et l'audit ; un moteur qui prédit sans encore décider (ses prédictions sont seulement écrites dans les reçus, pour être comparées aux décisions existantes) ; le moteur qui décide ; l'extension famille par famille de la table de règles ; enfin la sécurité et la sortie de l'ancien chemin par expressions

régulières. La partie C3 (le moteur PowerShell) est en cours de finalisation.

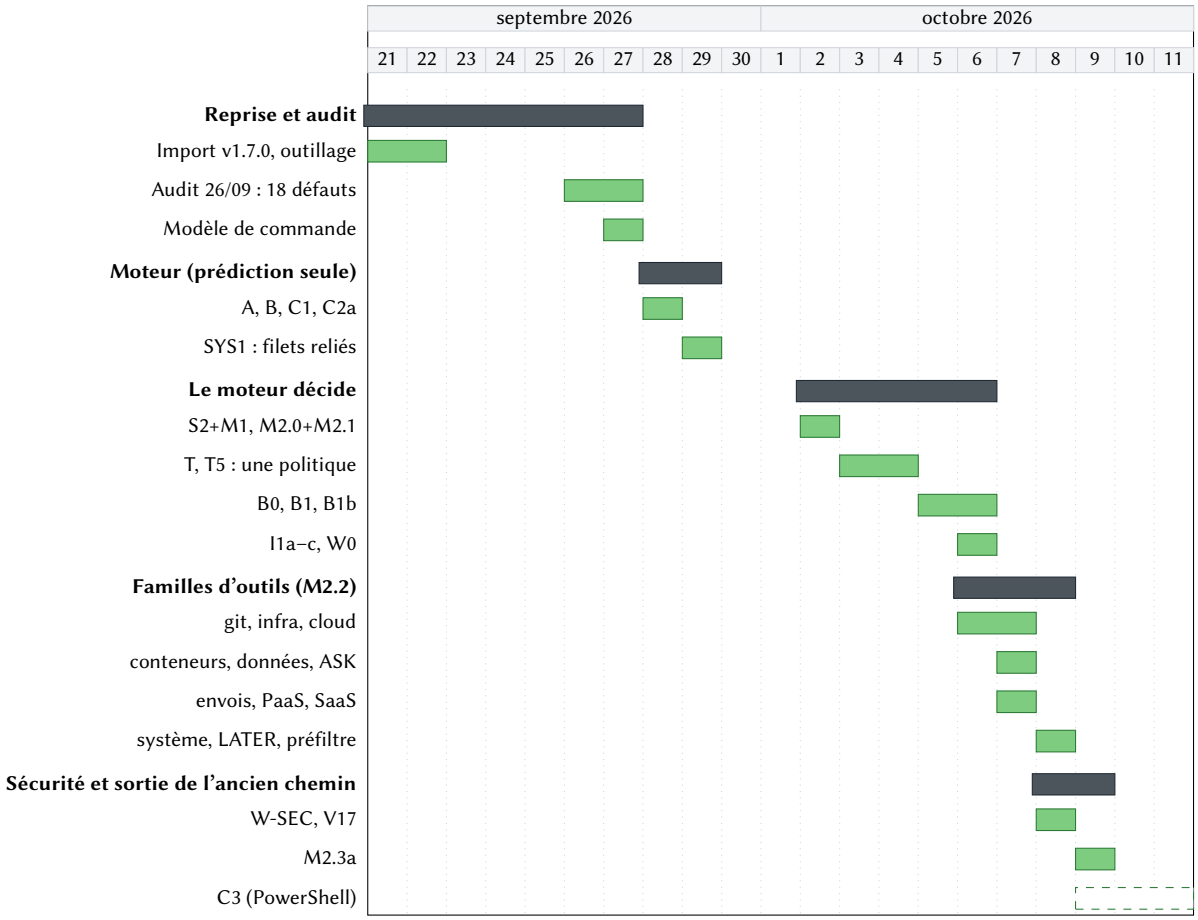



**Figure 1.2.** Chronologie des parties livrées, d'après l'historique git (21 septembre – 9 octobre 2026)

> **Pourquoi un moteur « en ombre » avant de le laisser décider ?**
>
> Les parties B à C2a ont livré un moteur qui *ne décidait rien* : il écrivait sa prédiction dans chaque reçu, et le portail vérifiait que zéro décision changeait. Pendant ce temps, chaque divergence entre la prédiction et la décision existante était un signal : soit un défaut du moteur, soit un défaut de l'ancien chemin. Le moteur n'a pris la main (partie S2+M1) qu'une fois ces divergences comprises, et ce changement-là a été revu ligne par ligne sur le portail.

## Conclusion du chapitre

Le stage part d'un outil qui existait, qui fonctionnait sur les cas simples et qui annonçait lui-même sa limite principale : il lisait les commandes avec des expressions régulières. Le problème est réel et documenté par des incidents publics ; les objectifs sont mesurables ; la méthode impose qu'aucune décision ne devienne plus permissive sans qu'on l'ait voulu. Le

chapitre suivant examine ce qui existe déjà pour protéger un développeur de son agent, et pourquoi ces protections ne suffisent pas.

CHAPITRE 2

# État de l'art et étude de l'existant

[À COMPLÉTER : chapitre 2 : rédigé après validation du chapitre 1]

CHAPITRE 3

# Analyse et conception

[À COMPLÉTER : chapitre 3 : texte à venir ; les figures ci-dessous sont définitives]

## 3.1 Modèle de menace et invariant

[À COMPLÉTER]

## 3.2 Architecture d'ensemble

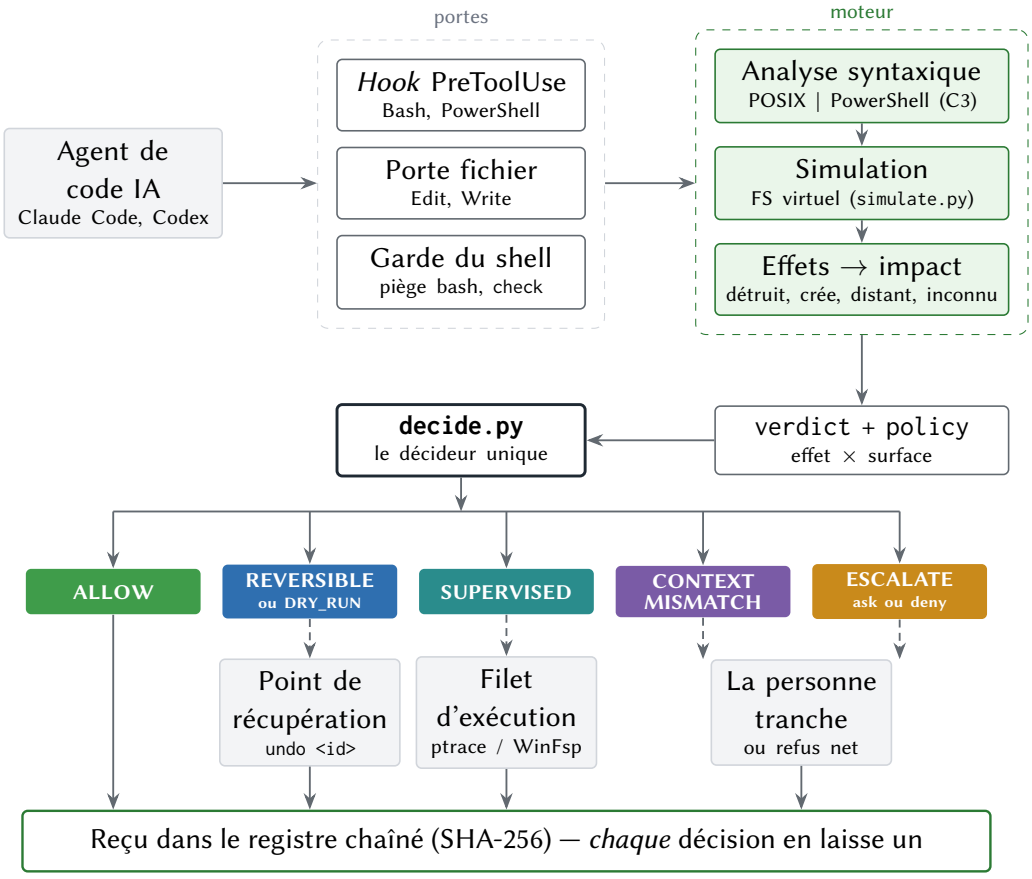



**FIGURE 3.1.** Le flux d'ensemble : de la commande de l'agent au reçu

| | Dans le projet | Sur la machine, hors du projet | Surfaces distantes<br>base, cloud, git, SaaS |
|---|---|---|---|
| **Lire avant**<br>le moteur (statique) | prédit ce qui est détruit, créé, déplacé | prédit la destruction → ESCALATE | reconnaît la surface et la sorte d'effet |
| **Récupérer**<br>instantané, point de contrôle | copie exacte avant, tout ou rien | aucune copie (V17 : jamais prétendue) | aucune copie locale ; indice chez le fournisseur |
| **Surveiller**<br>filet : ptrace ou WinFsp | sauvegarde chaque fichier au moment où il est détruit | non capturé (V18, en cours) | invisible au filet local |
| **Refuser**<br>plan de contrôle | `.demo_cli/`, `.demo_cli.toml`, *hooks* du projet | chemins listés seulement : `~/.bashrc`, *hooks*, tâche | sans objet |
| **Demander**<br>la personne (ask) | seulement si rien n'a pu être capturé | destruction hors projet : ask | ask ou deny selon la surface |
| **Tracer**<br>les reçus | chaque décision | chaque décision | chaque décision (sans secrets) |

couvert partiel non couvert — dit tel quel dans la raison de la décision

**Figure 3.2.** Ce que l'outil voit : chaque couche et ce qu'elle couvre, selon l'endroit où la commande agit

## 3.3 Conception du moteur

## 3.4 La politique et le décideur unique

## 3.5 Deux systèmes, un décideur

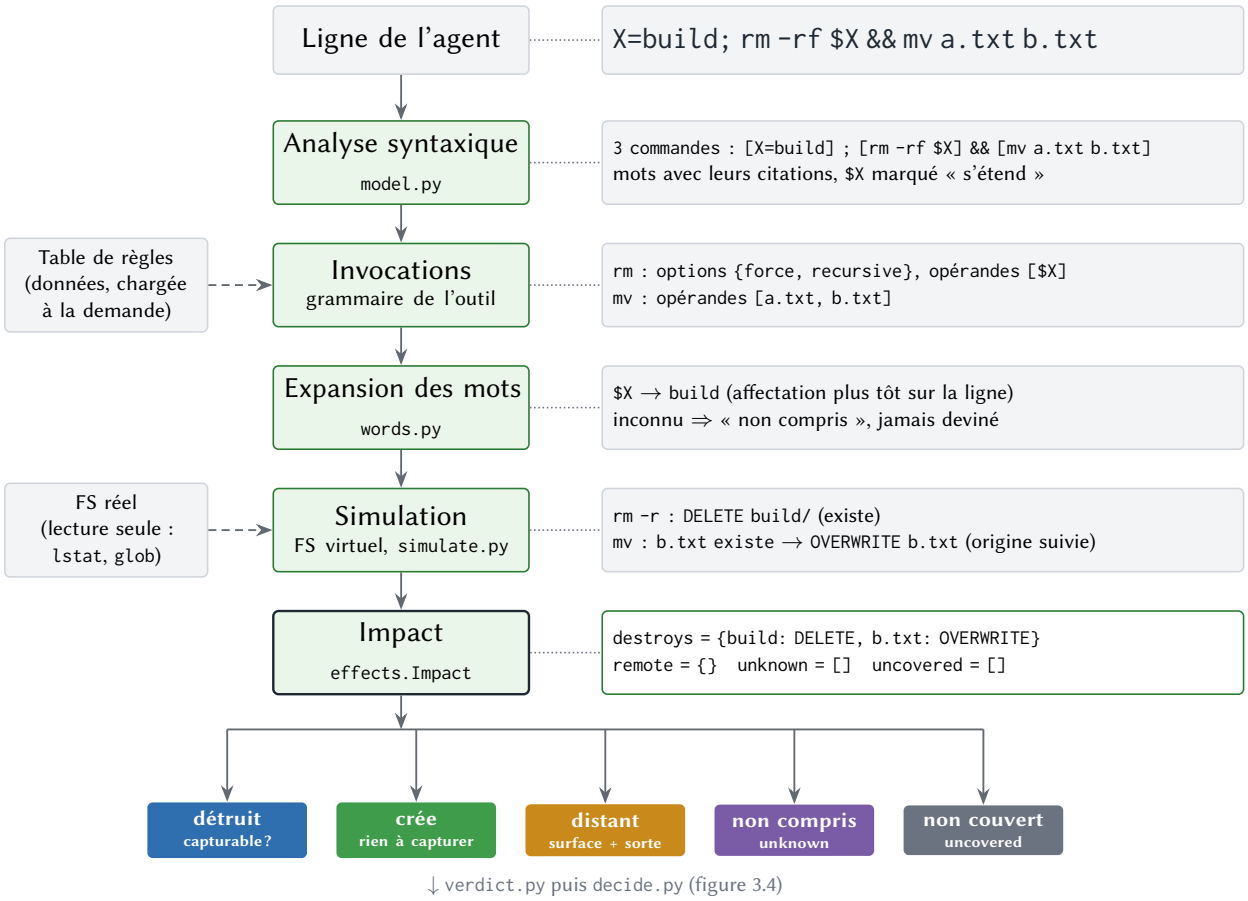



**FIGURE 3.3.** L'intérieur du moteur, sur un exemple réel (sortie du moteur au commit 3c94414)

| | **Ajout**<br>CREATE, APPEND, SERVICE_KEEP | **Perte**<br>DELETE, OVERWRITE, TRUNCATE, MOVE, SERVICE_LOSS | **Irrévocable**<br>SEND, RUN | **Plus tard**<br>LATER |
|---|---|---|---|---|
| **Dans le projet**<br>fichiers, base sqlite | ALLOW<br>rien d'existant détruit | REVERSIBLE<br>capture exacte, tout ou rien ; sinon ESCALATE·ask | SUPERVISED<br>code non lu : filet, sinon point de contrôle, sinon ALLOW + note | ESCALATE · ask<br>hook git, .envrc… |
| **Hors du projet**<br>même machine | ALLOW<br>rien d'existant détruit | ESCALATE · ask<br>aucune copie possible (V17) | ALLOW<br>+ note « rien capturé » : le filet ne voit pas hors projet (V18) | ESCALATE · ask<br>~/.bashrc, crontab, tâche… |
| **Fichiers de `demo_cli`**<br>plan de contrôle | ESCALATE · deny<br>même une création | ESCALATE · deny | ESCALATE · deny | ESCALATE · deny |
| **Surface « ask »**<br>base distante, git distant, bucket, conteneur… | ALLOW<br>listé dans le reçu | ESCALATE · ask<br>indice : PITR, versions, reflog… | ESCALATE · ask<br>ssh hôte cmd, publication… | ESCALATE · ask<br>`deferred_execution` |
| **Surface « deny »**<br>infra détruite, flush, disque, paiement… | ALLOW<br>listé dans le reçu | ESCALATE · deny<br>tout part, pas de retour réaliste | ESCALATE · deny<br>paiement, file purgée… | – |

**Ligne non comprise :** filet → point de contrôle → ESCALATE · ask **Télécharger-et-exécuter :** ESCALATE · deny

**Cible déclarée « production » :** capture, puis CONTEXT_MISMATCH montré à la personne, jamais approuvable sur place.

**FIGURE 3.4.** La table de décision : où l'effet tombe × quelle sorte d'effet

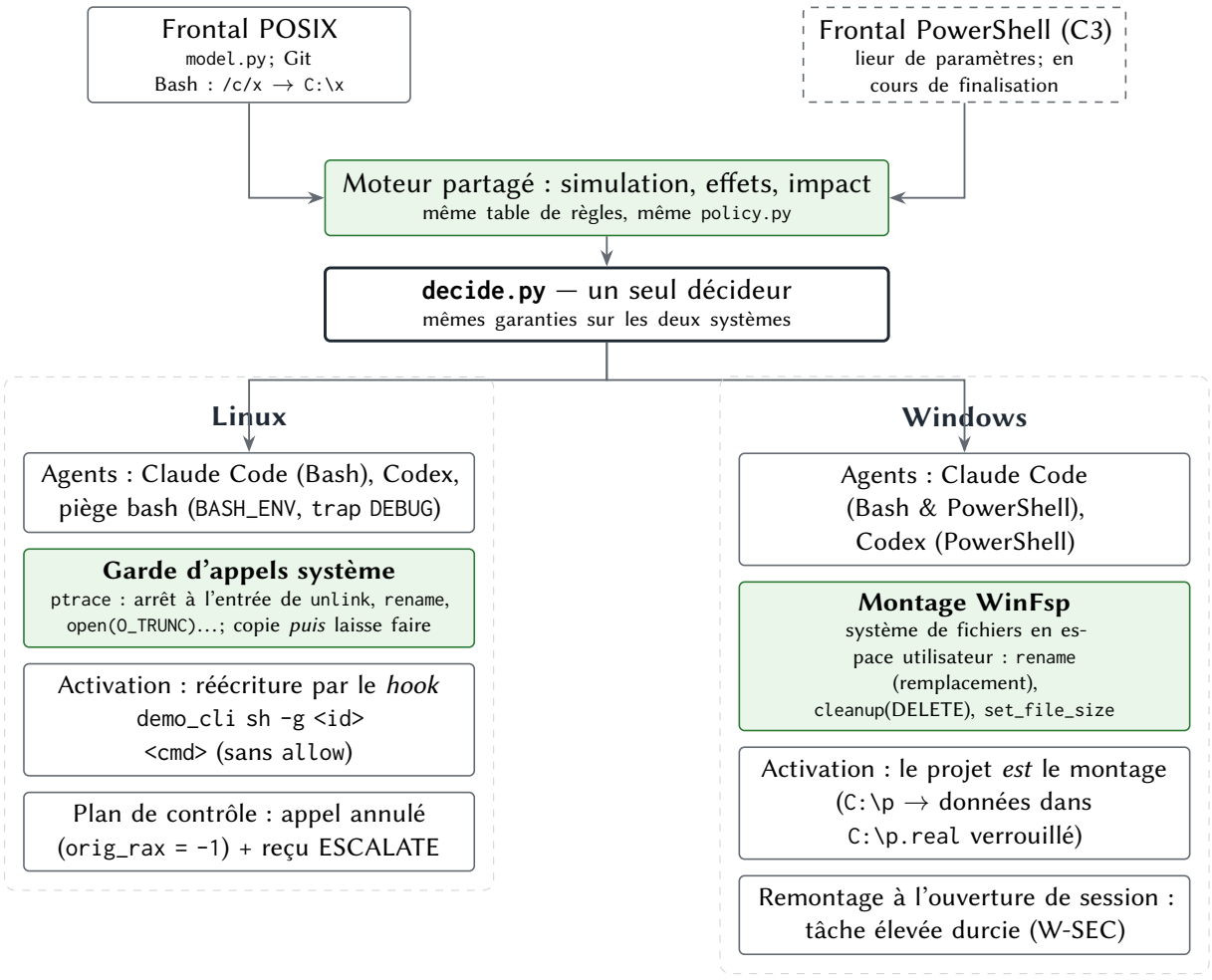



**Figure 3.5.** Linux et Windows : un seul décideur, deux filets d'exécution

CHAPITRE 4

# Réalisation

**[À COMPLÉTER : chapitre 4 : rédigé après validation du chapitre 3]**

## 4.1 Le côté Windows et C3, le second frontal du moteur

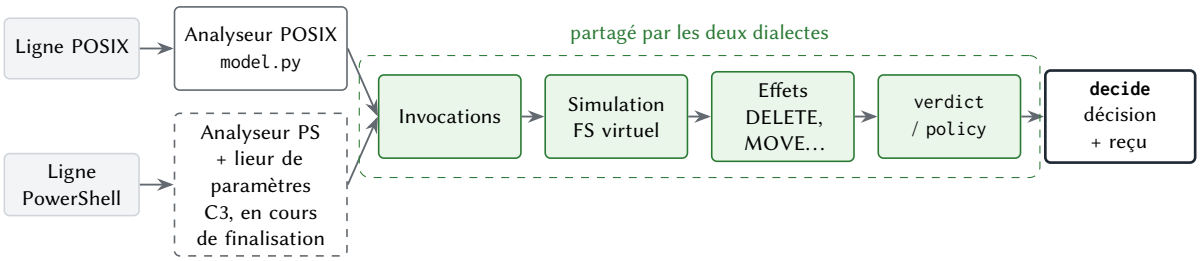



**FIGURE 4.1.** C3 : deux frontaux, un seul moteur (d'après le brief C3 ; statut : en cours de finalisation)

CHAPITRE 5

# Sécurité, validation et limites

[À COMPLÉTER : chapitre 5 : rédigé après validation du chapitre 4]

# Conclusion générale et perspectives

[À COMPLÉTER : conclusion et perspectives]

# Bibliographie et webographie


[1] WePwn. *WePwn — site officiel*. URL : https://wepwn.ma/ (visité le 09/10/2026).

[2] WePwn. *WePwn Access — portail des chercheurs*. URL : https://access.wepwn.ma/ (visité le 09/10/2026).

[3] WePwn. *Services : divulgation de vulnérabilités gérée*. URL : https://www.wepwn.ma/Services (visité le 09/10/2026).

[4] WePwn. *demo_cli, version 0.4.0-beta.8*. Licence MIT. 19 juill. 2026. URL : https://github.com/WePwn/demo_cli (visité le 09/10/2026).

[5] OECD.AI. *Replit AI Coding Tool Deletes Live Production Database and Fakes Data*. AI Incidents Monitor. 19 juill. 2025. URL : https://oecd.ai/en/incidents/2025-07-19-1eb1 (visité le 09/10/2026).

[6] eWeek. *'Catastrophic Failure' : AI Agent Wipes Production Database, Then Lies About It*. Juill. 2025. URL : https://www.eweek.com/news/replit-ai-coding-assistant-failure/ (visité le 09/10/2026).

[7] AI Incident Database. *Incident 1178 : Gemini CLI deletes a user's files after a failed directory creation*. 21 juill. 2025. URL : https://incidentdatabase.ai/cite/1178 (visité le 09/10/2026).

[8] Anuraag Gupta. *google-gemini/gemini-cli, issue 4586*. Juill. 2025. URL : https://github.com/google-gemini/gemini-cli/issues/4586 (visité le 09/10/2026).

[9] anthropics/claude-code. *Issue 76626 : the agent deletes the folder contents while attempting a file count*. 11 juill. 2026. URL : https://github.com/anthropics/claude-code/issues/76626 (visité le 09/10/2026).

ANNEXE A

# Exemples de reçus

[À COMPLÉTER]

ANNEXE B

# Extrait de la table de règles

[À COMPLÉTER]

ANNEXE C

# Extraits de tests

[À COMPLÉTER]

ANNEXE D

# Captures d'écran

**[À COMPLÉTER : liste des captures, fournie avec le chapitre 4]**